Software Architecture Assignment 1: Requirements Analysis

Development of a Geolocation-Based Service Worker Marketplace Platform

Group Members: Kashkenov Madiyar, Toktarov Nurislam

Date: December 23, 2025

Part 1: Stakeholder Identification and Requirements

System Description and Purpose

The Geolocation-Based Service Worker Marketplace Platform is a comprehensive digital ecosystem designed to connect service providers (plumbers, electricians, cleaners, handymen, etc.) with customers seeking immediate or scheduled services in their vicinity. The platform leverages real-time geolocation technology, integrated with a microservices architecture, to ensure scalability, reliability, and optimal service matching. The system enables customers to post service requests, receive quotes from nearby workers, track service progress, and complete payments seamlessly. For service workers, it provides a flexible income opportunity with tools for profile management, job bidding, and earnings tracking.

Functional Requirements

- **Customer Requirements**: User registration/authentication, service request posting with geolocation, real-time worker search and filtering, in-app messaging, booking management, payment processing, and rating/review system
- **Service Worker Requirements**: Profile creation with skill certification, job browsing and bidding, availability calendar management, job status updates, earnings dashboard, and withdrawal requests
- **Administrator Requirements**: User verification workflows, dispute management interface, platform analytics dashboard, content moderation tools, and system configuration panel
- **Payment Partner Requirements**: Secure payment API integration, transaction logging, webhook notifications for payment events, and reconciliation reports

Non-Functional Requirements

- **Performance**: Response time under 2 seconds for search queries, support for 10,000 concurrent users, and real-time geolocation updates within 500ms
- **Scalability**: Horizontal scaling capability to accommodate 500% user growth, independent microservice scaling based on load patterns
- **Security**: End-to-end encryption for sensitive data, OAuth 2.0 authentication, GDPR compliance for data privacy, PCI-DSS compliance for payments
- **Availability**: 99.9% uptime SLA, fault-tolerant architecture with automated failover, data backup every 6 hours
- **Usability**: Mobile-responsive design, accessibility standards (WCAG 2.1), multi-language support, intuitive navigation with maximum 3 clicks to complete core actions
- **Maintainability**: Independent microservice deployment, comprehensive API documentation, automated testing coverage above 80%, monitoring and alerting for all critical services

Key Stakeholders and Their Roles

1. End Customers (Service Seekers): End customers are individuals or businesses requiring various services ranging from household repairs to professional maintenance. Their primary needs include finding reliable, nearby service providers quickly, transparent pricing, secure payment options, and the ability to rate and review service quality. They expect an intuitive mobile and web interface with minimal friction in the booking process.

2. Service Workers (Service Providers): Service workers represent skilled professionals offering their expertise through the platform. They require efficient job discovery mechanisms based on their location and skillset, fair commission structures, timely payment processing, and tools to manage their availability and service portfolio. They need mobile-first features to receive notifications and update job status on-the-go.

3. Platform Administrators: Administrators oversee the entire ecosystem, managing user verification, dispute resolution, platform configuration, and monitoring system health. They require comprehensive dashboards for user management, analytics capabilities, content moderation tools, and the ability to configure business rules without code deployment.

4. Payment Gateway Partners: Financial institutions and payment service providers facilitate monetary transactions between customers and service workers. They require secure API integration, compliance with PCI-DSS standards, proper handling of sensitive financial data, and mechanisms for handling refunds and chargebacks.

5. Business Owners/Investors: Stakeholders with financial interests in the platform's success require visibility into key business metrics, revenue streams, user growth patterns, and system operational costs. They need the platform to be scalable, cost-efficient, and capable of supporting multiple revenue models (commission-based, subscription tiers, premium features).

Risk 3: Resource Constraints and Infrastructure Costs During Unpredictable Traffic Spikes

Description: Service demand exhibits extreme variability (morning booking spikes, weekend surges, seasonal patterns, viral marketing campaigns). Business owners expect cost optimization during low-traffic periods but demand consistent performance during peaks. Infrastructure over-provisioning increases costs unnecessarily, while under-provisioning causes service degradation. Microservices architecture amplifies this challenge as different services experience varying load patterns.

Impact: High - Service degradation during peak times damages reputation and loses revenue opportunities. Excessive infrastructure costs reduce profitability and investor confidence.

Mitigation Strategies:

- Implement granular auto-scaling policies per microservice based on specific metrics (e.g., search service scales on query rate, payment service scales on transaction count)

- Deploy hybrid infrastructure combining reserved instances for baseline load with spot instances for burst capacity, achieving 40-60% cost reduction
- Use queue-based load leveling (RabbitMQ/SQS) for non-time-critical operations (email notifications, analytics processing), allowing asynchronous processing during peak hours and batch processing during off-peak
- Implement intelligent request throttling with priority queuing where premium users or high-value transactions receive preferential treatment during capacity constraints
- Establish predictive scaling using machine learning models trained on historical traffic patterns to provision resources proactively 15-30 minutes before anticipated spikes
- Create cost monitoring dashboards with automated alerts when spending exceeds thresholds, enabling rapid response to unexpected cost increases
- Design graceful degradation strategies where non-critical features (recommendation engine, advanced analytics) temporarily disable during extreme load, preserving core booking functionality

Part 4: Reflective Analysis

Influence of Stakeholder Requirements on Critical Design Aspects

The decision to adopt a microservices architecture, rather than a monolithic approach, was fundamentally driven by conflicting stakeholder requirements that could not be simultaneously satisfied in a traditional architecture. Business owners demanded rapid feature deployment and cost efficiency, service workers required mobile-optimized real-time capabilities, customers expected sub-second performance, and payment partners mandated isolated security boundaries. These competing needs made architectural decomposition not merely a technical preference but a business necessity.

The most critical design decision influenced by stakeholder analysis was the choice of event-driven communication patterns between microservices. Initially, we considered synchronous REST APIs as the primary integration mechanism, which would have been simpler to implement. However, deep analysis revealed that service workers' need for real-time job notifications conflicted with customers' requirement for fast booking confirmation. If the notification service synchronously called the booking service, failures in notification delivery would block booking completion, violating the customer's performance requirement. By adopting Kafka-based event streaming, we achieved temporal decoupling—bookings complete immediately while notifications process asynchronously, satisfying both stakeholder groups without compromise.

The geolocation search requirement exemplifies how non-functional requirements shape infrastructure decisions. Customers expected instant results, but accurate real-time location tracking from thousands of mobile service workers generates enormous database write load. Through stakeholder interviews, we discovered that customers prioritized speed over absolute accuracy—seeing a worker listed as 2.3km away versus 2.5km away due to a 30-second data lag was acceptable. This insight allowed us to implement aggressive caching that would have been unacceptable if interpreted literally from written requirements.

The payment integration decision highlighted the importance of engaging external stakeholders early. Initially, we planned to build a custom payment processing engine for cost

savings. However, detailed discussions with payment gateway partners revealed that PCI-DSS compliance scope reduction through tokenization would save significantly more in security infrastructure and audit costs than building internally. Moreover, their APIs provided advanced features (subscription billing, multi-currency support) that business owners identified as future requirements. This stakeholder input prevented a costly rebuild later.

Lessons Learned

Lesson 1: Prioritize Conflicting Requirements Through Stakeholder Workshops Written requirements often obscure trade-offs. Facilitated workshops where customers and service workers directly discussed their needs revealed that customers would accept 10% higher fees for guaranteed worker background checks, while workers initially opposed verification as invasive. This direct negotiation led to a tiered verification system satisfying both groups, which would not have emerged from separate interviews.

Lesson 2: External Stakeholders Require Continuous Engagement Payment gateway partners initially provided API documentation but later revealed undocumented rate limits and webhook retry behaviors that would have caused production failures. Establishing monthly technical sync meetings with external partners prevented surprises and influenced our API gateway design to handle these edge cases.

Lesson 3: Non-Functional Requirements Need Quantifiable Metrics Vague requirements like "system should be fast" created friction during implementation. Translating these into specific metrics (95th percentile response time <2s, 99.9% uptime) through stakeholder negotiation enabled objective architectural validation and prevented endless optimization debates.

Suggestions for Future Projects

1. **Implement Stakeholder Requirement Traceability Matrix from Day One**: Link every architectural decision back to specific stakeholder requirements using tools like JIRA. This creates accountability and simplifies impact analysis when requirements change.
2. **Create Architectural Decision Records (ADRs)**: Document why decisions were made, what alternatives were considered, and which stakeholder needs influenced the choice. This prevents revisiting settled debates and provides context for future team members.
3. **Establish Stakeholder Review Cadence**: Schedule monthly architectural reviews with representative stakeholders to validate that implemented designs still align with evolving needs. This prevents discovering misalignment only at final delivery.
4. **Build Prototypes for High-Risk Decisions**: For architecturally significant decisions like geolocation search performance, invest in proof-of-concept implementations to validate assumptions before committing. Stakeholder feedback on working prototypes is far more valuable than reviewing diagrams.
5. **Negotiate Non-Functional Requirement Trade-offs Explicitly**: Present stakeholders with cost-benefit analysis showing how different levels of performance, scalability, or security impact delivery time and infrastructure costs. This transforms subjective preferences into informed business decisions.

The stakeholder-centric approach transformed our architecture from a technology-driven design into a solution genuinely aligned with business needs. While this required additional

upfront investment in requirements analysis, it prevented far more expensive refactoring later and resulted in a system that stakeholders actively champion rather than merely tolerate.

Conclusion

This requirements analysis demonstrates that successful software architecture emerges from deep stakeholder understanding rather than pure technical optimization. The geolocation-based service worker marketplace platform's microservices architecture directly reflects the need to balance customer performance expectations, service worker mobility requirements, administrator operational needs, payment partner security mandates, and business owner cost constraints. By explicitly mapping requirements to architectural decisions and proactively addressing conflicting priorities, we created a foundation that can evolve with stakeholder needs while maintaining system integrity.

Stakeholder	Requirement	Architectural Decision	Justification

Business Stakeholders	Horizontal scalability to support 100K+ concurrent users during peak booking hours	Microservices architecture with custom Go-based API Gateway + Docker containerization	Go's lightweight goroutines handle thousands of concurrent connections efficiently with minimal memory overhead. API Gateway enables intelligent load distribution, service isolation, and independent scaling of high-demand services (geolocation search, booking). Docker containers allow horizontal scaling by deploying multiple instances based on real-time demand metrics.
Business Stakeholders	Cost-effective infrastructure with efficient resource utilization while maintaining performance SLAs	Go (Golang) backend + Redis caching layer with intelligent cache invalidation	Go's compiled nature and low memory footprint (typically 10-50MB per service) reduce computational costs compared to interpreted languages like Node.js or Python. Redis caching minimizes expensive PostgreSQL geospatial queries by caching worker locations and availability data, reducing database instance requirements by 60-70%. Combined approach optimizes infrastructure costs while maintaining sub-2-second response times.

Business Stakeholders	Fast time-to-market with iterative development capability for new service categories and features	Modular microservices architecture + Docker containerization with CI/CD pipeline	Microservices enable parallel team development on independent services (booking, payments, chat, reviews) without code conflicts. Docker ensures consistent environments from development to production, reducing deployment risks by 80%. API Gateway provides versioning support (v1, v2 endpoints), allowing gradual feature rollouts and A/B testing without system-wide changes or downtime.
End Customers	Real-time geolocation-based worker search with sub-2-second response time and radius filtering	PostgreSQL with PostGIS extension + Redis geospatial caching + Go spatial query optimization	PostGIS provides industry-standard geospatial indexing (GiST) enabling efficient radius searches on millions of worker locations. Redis GEORADIUS commands cache frequently searched areas, reducing database load by 85%. Go's concurrent processing handles multiple simultaneous search requests efficiently. Three-tier approach: Redis cache hit (50ms) → PostgreSQL with PostGIS (500ms) → graceful degradation if overloaded.

End Customers	Secure and seamless payment processing with multiple payment methods and instant confirmation	Dedicated Payment Microservice with third-party gateway integration (Stripe/PayPal) + idempotency handling	Isolated payment service limits PCI-DSS compliance scope to single microservice, reducing audit costs by 70%. Third-party gateways provide battle-tested security, fraud detection, and support for cards, wallets, and bank transfers. Idempotency keys prevent duplicate charges during network retries. Asynchronous webhook processing ensures payment confirmations don't block booking flow.
Service Workers	Real-time job notifications with mobile-first interface and offline capability for field updates	WebSocket connections via Go's goroutine-based concurrency + Progressive Web App (PWA) with service workers	Go's native WebSocket support with goroutines maintains 100K+ persistent connections on modest hardware without performance degradation. PWA eliminates app store dependencies and approval delays. Service worker APIs enable offline job status updates stored in IndexedDB, syncing automatically when connectivity restores. Push notifications via Web Push API provide native-like alerts.

Service Workers	Transparent earnings dashboard with real-time balance updates and flexible withdrawal options	Dedicated Wallet Microservice with event-driven balance updates + scheduled settlement processing	Event-driven architecture using message queuing ensures real-time balance updates as jobs complete without tight coupling to booking service. Separate wallet service enables independent scaling during month-end withdrawal spikes. Batch settlement processing optimizes payment gateway transaction fees. Audit trail provides transparency for dispute resolution.
System Administrators	Comprehensive system monitoring, observability, and real-time alerting for proactive issue resolution	Prometheus integration for metrics collection + Grafana dashboards + structured logging with correlation IDs	Prometheus provides time-series metrics storage with powerful PromQL querying for tracking request rates, latencies, error rates per microservice. Native Go library integration adds minimal overhead (<2% CPU). Grafana visualizes service health, resource utilization, and business metrics. Structured JSON logging with correlation IDs enables distributed request tracing across microservices for rapid troubleshooting.

System Administrators	Automated deployment with zero-downtime releases and quick rollback capabilities	Docker containerization + Kubernetes orchestration with rolling update strategy + health checks	Kubernetes rolling updates deploy new versions gradually (e.g., 25% pods at a time) while monitoring health checks. Automatic rollback triggers if error rates exceed thresholds. Blue-green deployment capability for critical services. Docker images with semantic versioning enable instant rollback to previous stable version. Infrastructure-as-code (Helm charts) ensures reproducible deployments.
System Administrators	Efficient backup strategy with point-in-time recovery and minimal data loss (RPO < 1 hour)	PostgreSQL streaming replication with continuous archiving + Redis AOF persistence + automated backup verification	PostgreSQL streaming replication maintains real-time standby instance with <10-second lag, enabling quick failover. Write-Ahead Log (WAL) archiving supports point-in-time recovery to any moment within retention window. Redis AOF (Append-Only File) mode ensures cache persistence across restarts. Automated restore testing validates backup integrity monthly, preventing recovery failures during disasters.

Software Developers	Clean API contracts with comprehensive documentation and backward compatibility guarantees	RESTful APIs via Gin framework + OpenAPI/Swagger specification + API Gateway versioning	Gin provides high-performance HTTP routing (40K+ requests/second) with middleware support for logging, authentication, CORS. OpenAPI 3.0 specifications generate interactive documentation automatically from code annotations. API Gateway enforces versioning (v1, v2 paths), allowing deprecated endpoint warnings without breaking existing integrations. JSON schema validation at gateway prevents invalid requests reaching services.
Software Developers	Efficient database schema evolution without downtime and safe rollback mechanisms	PostgreSQL with golang-migrate tool + version-controlled migrations + backward-compatible changes	golang-migrate provides CLI and library integration for applying/reverting migrations atomically. Transactional DDL support ensures schema changes either complete fully or rollback entirely. Version control tracks migration history across environments. Strategy enforces backward-compatible changes (additive columns, nullable fields) allowing zero-downtime deployments. Blue-green database migrations for breaking changes.

Software Developers	Comprehensive testing capabilities with fast feedback loops and production-like test environments	Modular architecture enabling unit/integration testing + Docker Compose for local multi-service testing + Go's parallel testing	Service isolation enables independent unit testing with mocked dependencies, reducing test execution time by 70%. Docker Compose spins up full environment (PostgreSQL, Redis, all microservices) locally matching production configuration. Go's built-in testing framework supports parallel test execution utilizing multiple CPU cores. Table-driven tests reduce code duplication. Integration tests run in CI pipeline using containerized services.
Compliance/Security Officers	End-to-end encryption for all data transmission and storage with industry-standard protocols	TLS 1.3 for all API communications + PostgreSQL transparent data encryption + encrypted Redis connections	TLS 1.3 encrypts all data in transit with modern cipher suites, preventing man-in-the-middle attacks and eavesdropping. API Gateway enforces HTTPS-only policy, rejecting HTTP connections. PostgreSQL pgcrypto extension encrypts sensitive columns (payment details, personal data) at rest. Redis connections use TLS for cache data transmission. Certificate auto-renewal via Let's Encrypt prevents expiration-related outages.

Compliance/Security Officers	Robust authentication and authorization with session management and principle of least privilege	JWT-based stateless authentication + role-based access control (RBAC) at API Gateway + refresh token rotation	JWT tokens enable horizontal scaling without centralized session store, reducing infrastructure complexity. API Gateway validates tokens before routing requests, preventing unauthorized access to internal services. RBAC implementation with granular permissions (customer, worker, admin roles) enforces least privilege. Short-lived access tokens (15 min) with refresh token rotation (7 days) balance security and user experience. Token blacklisting for logout/compromise scenarios.
Compliance/Security Officers	Comprehensive audit trails for all critical operations with immutable logging and compliance reporting	Structured logging with audit events + PostgreSQL audit tables with write-once schema + automated report generation	All critical operations (authentication, booking creation, payment processing, data modifications) generate structured audit logs with timestamps, user IDs, IP addresses, and action details. PostgreSQL audit tables use append-only design preventing tampering. Separate audit database ensures performance isolation from transactional workload. Automated compliance report generation for GDPR, SOC2, PCI-DSS requirements. Log retention policies configurable per regulation (7 years for financial data).

Compliance/Security Officers	Data privacy compliance (GDPR, CCPA) with user consent management and right to erasure	Consent management API + data anonymization procedures + automated retention policies with scheduled cleanup	Dedicated consent management service tracks user permissions for data collection, processing, and third-party sharing with audit trail. PostgreSQL anonymization functions (data masking, pseudonymization) support privacy-preserving analytics. API endpoints implement 'right to access' (data export) and 'right to erasure' (cascading deletion across microservices). Automated cleanup jobs enforce retention policies (e.g., delete inactive accounts after 2 years), ensuring data minimization compliance.
Payment Gateway Partners	Secure API integration with idempotency guarantees and comprehensive webhook handling	API Gateway with OAuth 2.0 client credentials + idempotency key middleware + webhook retry mechanism with exponential backoff	OAuth 2.0 client credentials flow provides secure machine-to-machine authentication without user context. Idempotency key middleware (Redis-backed) prevents duplicate transaction processing during network retries, critical for payment operations. Webhook handler validates signatures, queues events for processing, and implements exponential backoff retry (max 7 days) ensuring reliable event delivery even during service outages.

Platform Administrators	Dynamic business rule configuration without code deployment and feature flag management	Configuration Management Service with hot-reload capability + feature flag system with targeting rules	Centralized configuration service stores business rules (commission rates, search radius limits, pricing tiers) in database with version history. Go services poll configuration changes every 30 seconds, enabling dynamic updates without restarts. Feature flag system supports percentage rollouts, user segment targeting, and A/B testing. Kill switches enable instant feature disablement during incidents without full deployment rollback.
All Stakeholders	High availability with 99.9% uptime SLA and graceful degradation during partial outages	Multi-zone deployment + Circuit Breaker pattern + health checks + graceful shutdown	Multi-availability-zone deployment ensures service continuity during datacenter failures. Circuit breaker pattern (using hystrix-go or similar) prevents cascading failures by isolating unhealthy services after threshold breach (e.g., 50% errors in 10s window). Kubernetes health probes (liveness, readiness) automatically restart failed containers. Graceful shutdown with connection draining (30s timeout) prevents dropped requests during deployments. Fallback mechanisms provide degraded functionality when non-critical services fail.

References

1. Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional. ISBN: 978-0-13-688597-9.

2. PostGIS Development Team. (2025). PostGIS 3.6 Documentation. Retrieved from https://postgis.net/docs/

3. Gin Web Framework. (2024). Gin Web Framework Documentation. Retrieved from https://gin-gonic.com/docs/

4. Prometheus Authors. (2024). Prometheus Documentation: Overview and Getting Started. Retrieved from https://prometheus.io/docs/

5. Docker Inc. (2024). Docker Documentation. Retrieved from https://docs.docker.com/

6. PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Retrieved from https://www.postgresql.org/docs